Design Patterns

Deepraj S Dixit
dixit@ncst.ernet.in

Pattern Origins

- Christopher Alexander

Notes on the Synthesis of Form

Harvard University Press, 1964

The Oregon Experiment

Oxford University Press, 1975

A Pattern Language: Towns, Buildings, Construction

Oxford University Press, 1977

The Timeless Way of Building

Oxford University Press, 1979

Patterns History

- Cunningham & Beck OOPSLA-87
- Jim Coplien C++ Idioms
- Bruce Anderson OOPSLA-91/92
- Hillside Group
- PLoP Conferences
- Gang of Four

What is a Pattern?

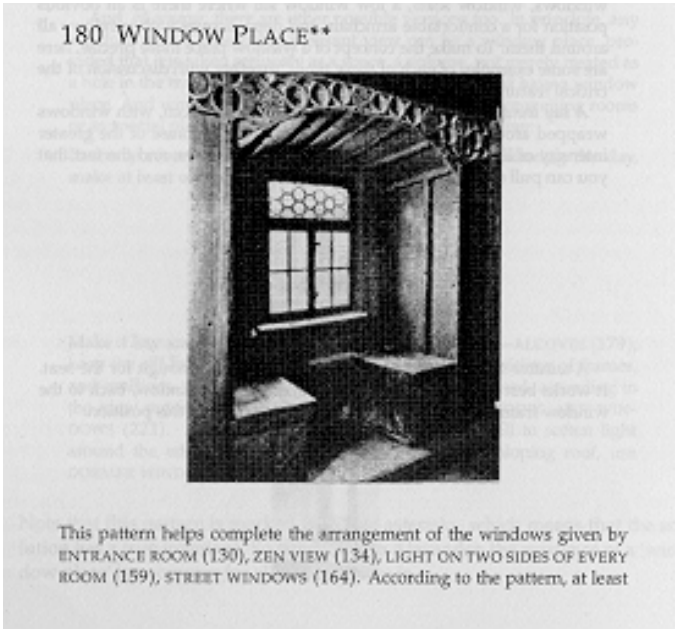
- A pattern is the abstraction from concrete form which keeps recurring in specific non-arbitrary contexts

Kinds of Patterns

- Towns, Buildings & Constructions: Chris
- Design Patterns: Gang of Four
- Analysis Patterns: Martin Fowler
- Organizational & Process Patterns: Jim Coplien
- Software Engineering Patterns
- Meta-Patterns, Anti-Patterns

Components of a Pattern

- Name
- Problem
- Context
- Forces
- Solution
- Examples
- Resulting Context
- Rationale
- Related Patterns
- Known Uses
- Abstract or Pattern Thumbnail


180 WINDOW PLACE**

This pattern helps complete the arrangement of the windows given by ENTRANCE ROOM (130), ZEN VIEW (134), LIGHT ON TWO SIDES OF EVERY ROOM (159), STREET WINDOWS (164). According to the pattern, at least

180. Window Place

- Everybody loves window seats, bay windows, and big windows with low sills and comfortable chairs drawn up to them
- In every room where you spend any length of time during the day, make at least one window into a “window place”

Classification of SW Patterns

- Architectural
- Design
- Idioms

or...

- Conceptual
- Design
- Programming

Design Patterns

Design Patterns :
Elements of Reusable Object-Oriented Software

– Erich Gamma, Richard Helm, Ralph Johnson, John Vlissides

LISP

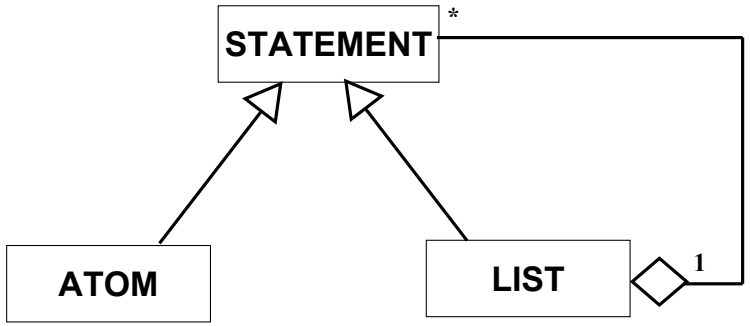
- ATOM and LIST

  12, “hello”, nil, (), (12), (12 (“hi”) ())

  ATOMS LISTS

- What is a relationship between ATOM and LIST?

Composite



Composite Pattern

- Category
  - Object Structural
- Intent
  - compose objects into tree structures to represent part-whole hierarchies
  - lets clients treat individual objects and compositions of objects uniformly

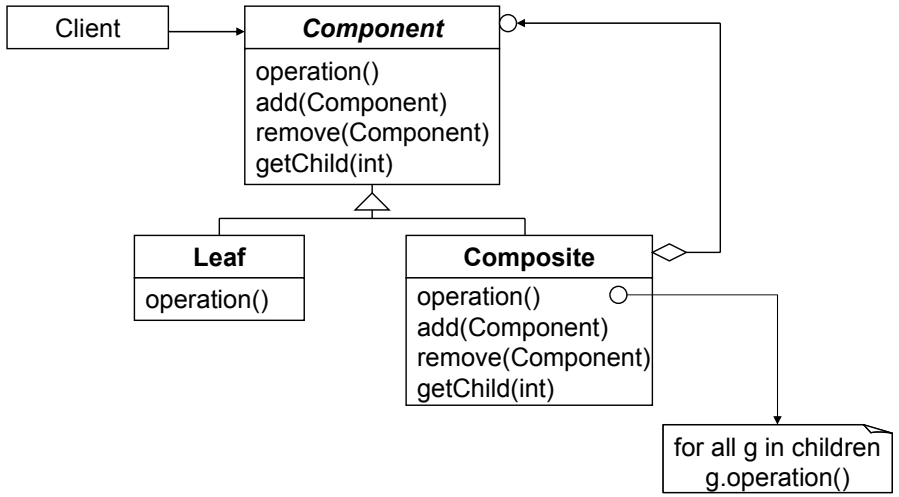
Composite

- Applicability
  - represent part-whole hierarchies of objects
  - clients to be able to ignore difference between compositions of objects and individual objects

Composite

- Consequence
  - defines class hierarchies consisting of primitive objects and composite objects
  - makes client simple
  - makes it easier to add new kinds of components
  - can make design overly general

Composite -- Structure



Applications of Composite

- Windowing System: Window and Panel
- File System: File and Directory
- Drawing Application: Shape and Group

- Wherever “nested object hierarchy” is required

Unique Object Instance

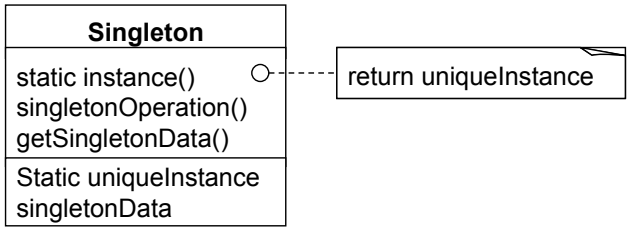
- Ensure a class only has one instance and provide a global point of access to it

- Say “the Application” object

Singleton

- Object Creational
- Applicability
  - there must be exactly one instance of a class
  - must be accessible to clients from a well-known access point
  - when sole instance should be extensible by subclassing and clients should be able to use subclass without modifying their code

Singleton -- Structure



Singleton

- Consequences
  - controlled access to sole instance
  - reduced name space
  - permits refinement of operations and representation
  - permits variable number of instances
  - more flexible than class operations

Singleton: Implementation

```
class Singleton {
   public :
        static Singleton* instance();
   protected :
        Singleton();
   private :
        static Singleton* _instance;
};
```


Singleton: Implementation

```
Singleton* Singleton::_instance = 0;

Singleton* Singleton::instance()
{
   if ( _instance == 0 )
        _instance = new Singleton;

   return _instance;
}
```


Structural Patterns

Class
- Adapter

Object
- Composite
- Facade
- Proxy
- Bridge
- Decorator
- Flyweight

Creational Patterns

Class
- Factory Method

Object
- Singleton
- Builder
- Abstract Factory
- Prototype

Behavioural Patterns

Class
- Interpreter
- Template Method

Object
- Command
- Chain of Responsibility

Object
- Iterator
- Observer
- State
- Visitor
- Mediator
- Strategy

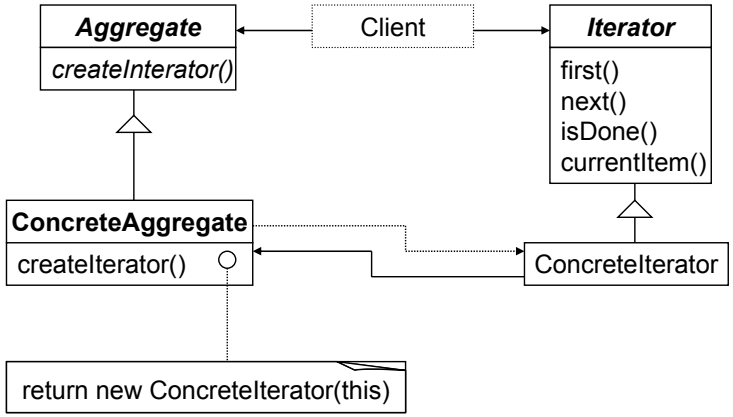
Iterator

- Category
  - Object Behavioural
- Intent
  - provide a way to access elements of an aggregate object sequentially without exposing its underlying representation
- Also Known As
  - Cursor

Iterator

- Applicability
  - access an aggregate object's content without exposing internal representation
  - support multiple traversals of aggregate objects
  - provide uniform interface for traversing aggregate structures

Iterator -- Structure



Iterator

- Consequences
  - supports variations in traversal of an aggregate
  - simplify aggregate interface
  - more than one traversal can be pending on an aggregate

Observer

- Category
  - Object Behavioural
- Intent
  - define one-to-many dependency between objects
  - when one object changes state, all its dependents are notified and updated automatically

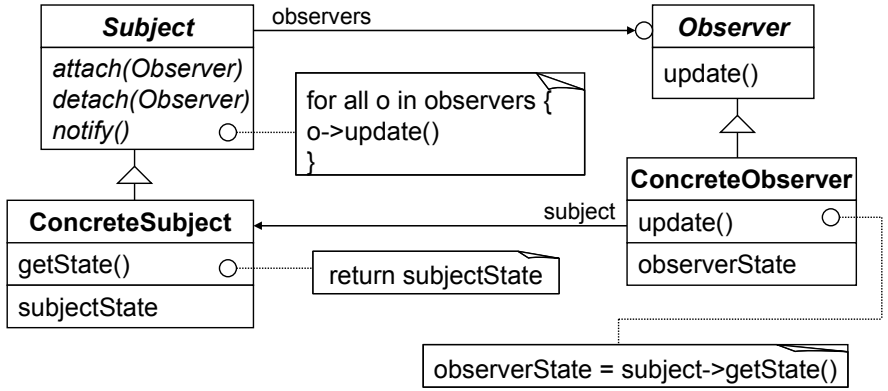
Observer

- Also Known As
  - Dependents, Publish-Subscribe

Observer

- Applicability
  - abstraction has two aspects, one dependent on the other
  - change to one object requires changing others and number of objects to be changed is not known
  - object should be able to notify other objects without making assumptions who they are

Observer -- Structure



Observer

- Consequences
  - abstract coupling between subject and observer
  - support for broadcast communication
  - unexpected updates

Simple Graphics Package

- Geometric Shapes

  Text, Line, Rectangle, Circle, Image, …
- Operations

  Save, Load, Create, Move, Rotate, Delete, Copy, Paste, Group, Edit, Macros, …
- Typical GUI
  - Menu, Canvas, Toolbar, Property Sheets, …

SGP: Design Issues

- Extensible set of Shapes
- What about Groups of Shapes?
- How to handle Undo & Redo?
- What about Macros?
- Smart Canvas: Updates & Tools
- Loading Shapes from File
- Extensible set of Operations

Extensible Shapes

- Inheritance

Groups of Shapes

- Composite Shape
- Can use Iterator for traversal

Undo & Redo

- Operation as Command

Macros

- Composite Command!

Smart Canvas: Updates

- Shape as Subject
- Canvas and PropertySheet as Observer

Smart Canvas: Tools

- Tool as State and Canvas as Context

Loading Shapes from File

- Factory Method or Virtual Constructor

Extensible Operations

- Operations as Visitors

Pattern Catalogs

- A collection of related patterns, perhaps only loosely or informally related.
  - It typically subdivides the patterns into at least a small number of broad categories and may include some amount of cross referencing between patterns.

Pattern Systems

- A cohesive set of related patterns which work together to support the construction and evolution of whole architectures.
  - It is organized into related groups and subgroups at multiple levels of granularity, describes the many interrelationships between the patterns and their groupings and how they may be combined and composed to solve more complex problems.

Pattern Languages

- A structured collection of patterns that build on each other to transform needs and constraints into an architecture.
  - PL includes **rules** & **guidelines**, which explain how and when to apply its patterns to solve a problem which is larger than any individual pattern can solve.
  - Good PL is **generative**, capable of generating all the possible sentences from a rich and expressive pattern vocabulary.

Pattern on Pattern Mining

- Taking time to reflect
- Adhering to a structure
- Being concrete early
- Keeping patterns distinct and complementary
- Presenting effectively
- Iterating tirelessly
- Collecting and incorporating feedback

Conclusions

- *Program to an interface, not an implementation*
- *Favour object composition over class inheritance*
- Delegation as an underlying principle
- Use Design Patterns whenever possible

OO Future

- Synthesis of Patterns and Architectures
- Classification & indexing, tool support
- Multi-paradigm Design
- Role of semantic notation is crucial
  - UML is a good candidate

Parting Thought

It is possible to make buildings by stringing together patterns, in a rather loose way. A building made like this, is an assembly of patterns. It is not dense. It is not profound. But it is also possible to put patterns together in such a way that many patterns overlap in the same physical space: the building is very dense; it has many meanings captured in a small space; and through this density, it becomes profound.

-Christopher Alexander (A Pattern Language)

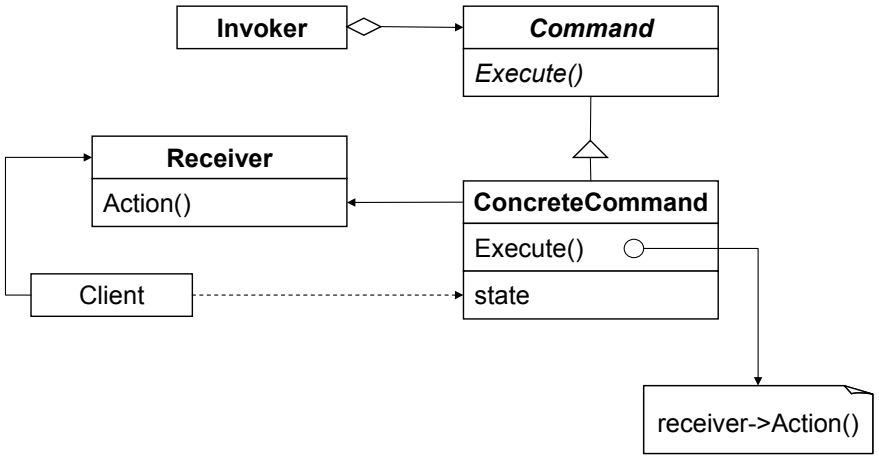
Supplementary Material …

Note: The following slides are not ordered for the lecture delivery.

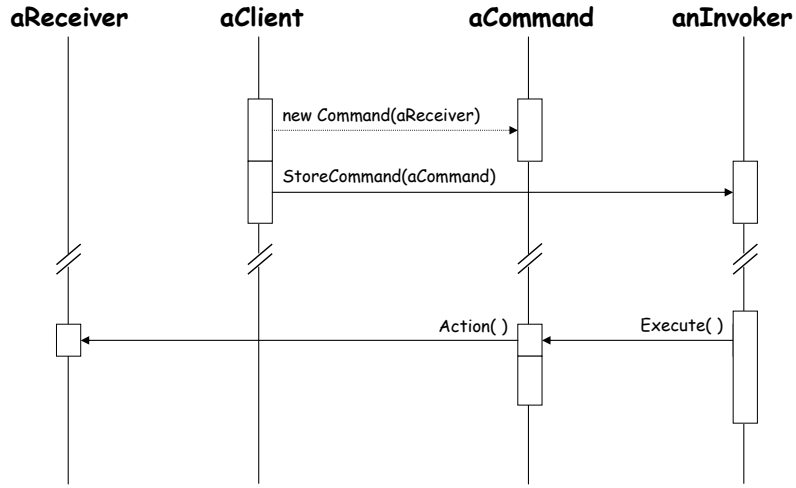
Command Pattern

- Category
  - Object Behavioural
- Intent
  - encapsulate a request as an object, thereby letting you parameterise clients with different requests, queue or log requests, and support undoable operations

Command -- Structure



Command Pattern



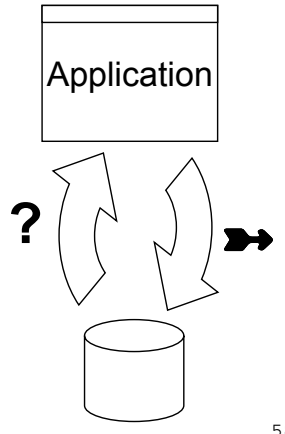
Command Pattern

- Consequence
  - decouples the object that invokes the operation from the one that knows how to perform it
  - first class objects
  - addition becomes simpler

De-serialize Data

- How to construct object structure while reading from a file?

```
Library ncst {
    Book ooad { author Booch; price R250 }
    Book gof { author Gamma; price $28 }
    Journal sigsoft {
        Issue v24n6 { conference ESEC99 }
        Issue v25n1 { conference ACM8SYM }
    }
    CD bullinfo { title info pages; issued KEY003 }
}
```



De-serialization

1. Read class name, say “Book”
2. Ask Creator to instantiate class $\Rightarrow$ *book*
3. Let *book* read the rest of record
   If the class can contains other class data, handle it in similar fashion
4. If any more data to be read, goto step 1

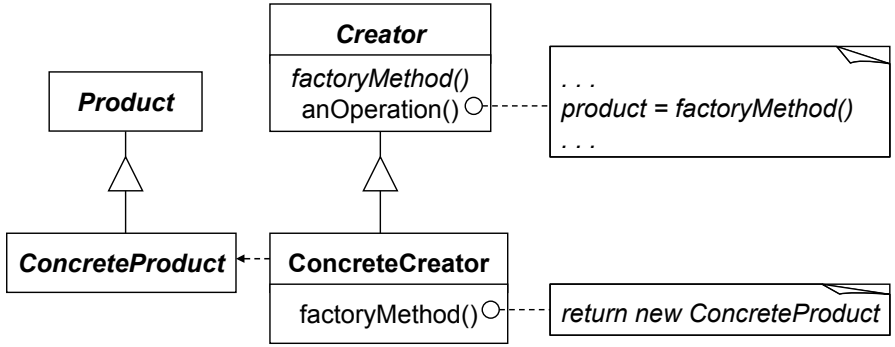
Factory Method

- Category
  - Class Creational
- Intent
  - define an interface for creating an object
  - let subclasses decide which class to instantiate
- Also Known As
  - Virtual Constructor

Factory Method

- Applicability
  - a class can't anticipate the class of objects it must create
  - a class wants its subclasses to specify objects it creates
  - classes delegate responsibility to one of several helper subclasses -- localize knowledge of which helper subclass is the delegate

Factory Method -- Structure



Simplifying Interface

- The application needs to compilation support for many languages. To that effect, a sophisticated class library is used. But, the sophistication is not needed and is creating understanding bottlenecks for programming team.

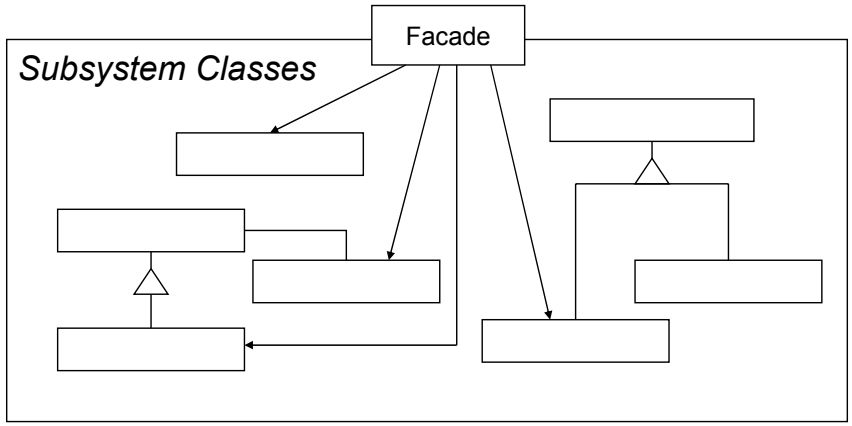
Facade

- Category
  - Object Structural
- Intent
  - provide a unified interface to a set of interface in a subsystem
  - defines a high-level interface that makes the subsystem easier to use

Facade

- Applicability
  - provide a simple interface to a complex subsystem
  - many dependencies between clients and implementation classes of an abstraction
  - layer the subsystem

Facade -- Structure



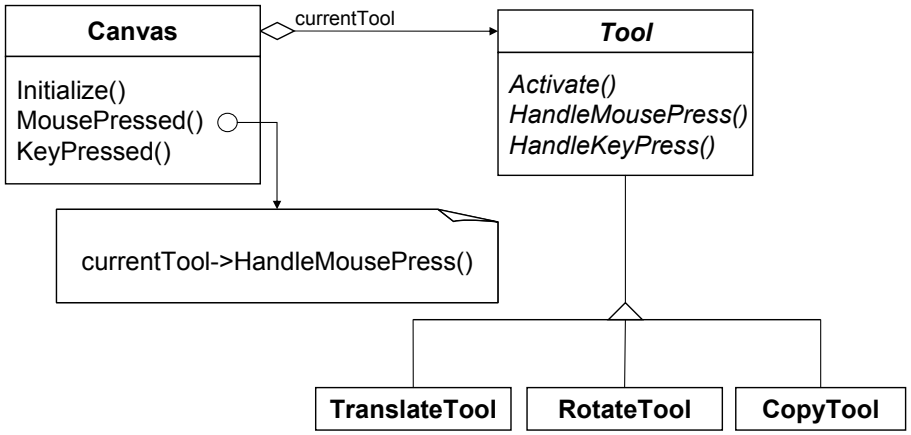
Facade

- Consequences
  - shields clients from subsystem components
  - promotes weak coupling between subsystem and its clients
  - doesn't prevent applications from using subsystem classes if they need to

State

- Category
  - Object Behavioural
- Intent
  - Allow an object to alter its behaviour when its internal state changes. The object will appear to change its class.

Tool -- Structure



State

- Consequence
  - localizes state-specific behaviour for different states
  - state transitions are made explicit
  - state objects can be shared

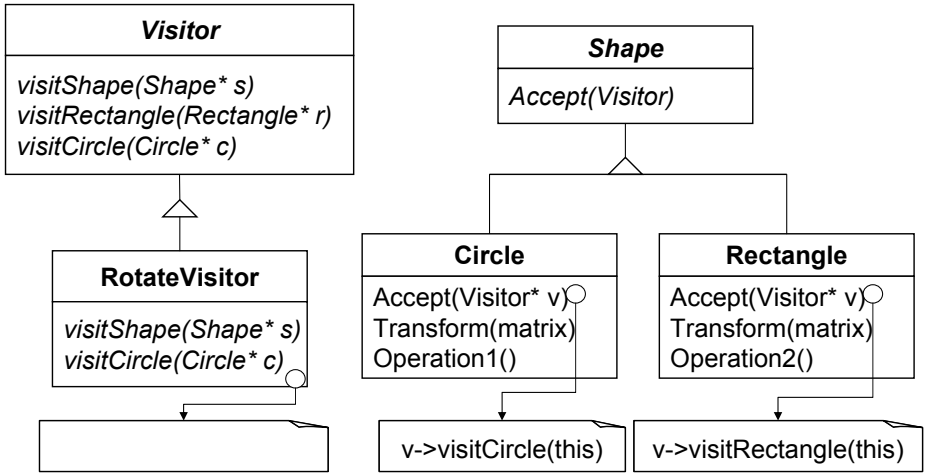
State

- Applicability
  - an object's behaviour depends on its state, and it must change its behaviour at run-time depending on that state
  - operations have large multipart conditionals that depend on the object's state, usually represented as an enumerated constant (or more)

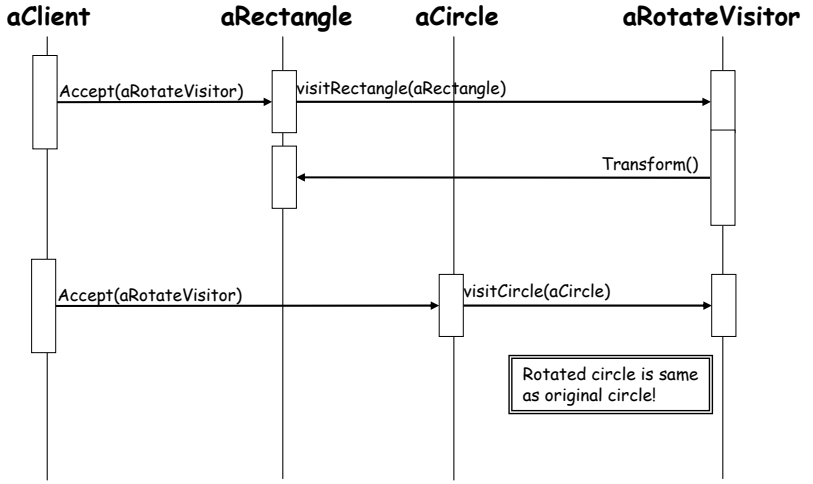
SGP: Applying Actions

- We have a stable shape hierarchy
- Users are not yet sure about the complete set of actions those should be supported by shapes. They feel, during development, actions will be discovered
- How are actions applied to shapes?

Visitor -- Structure



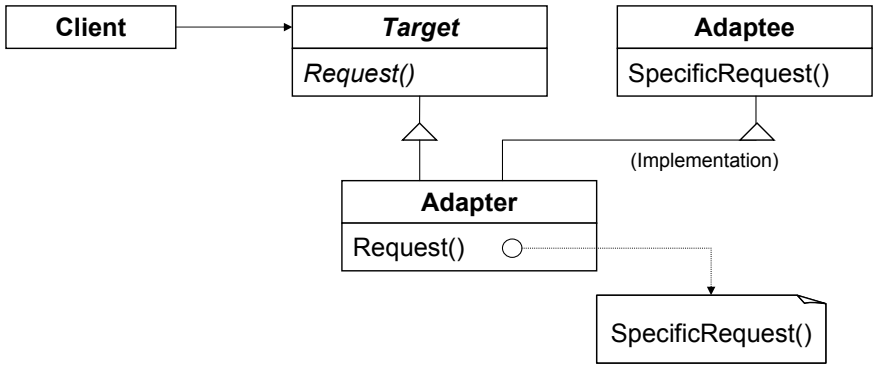
Visitor Pattern



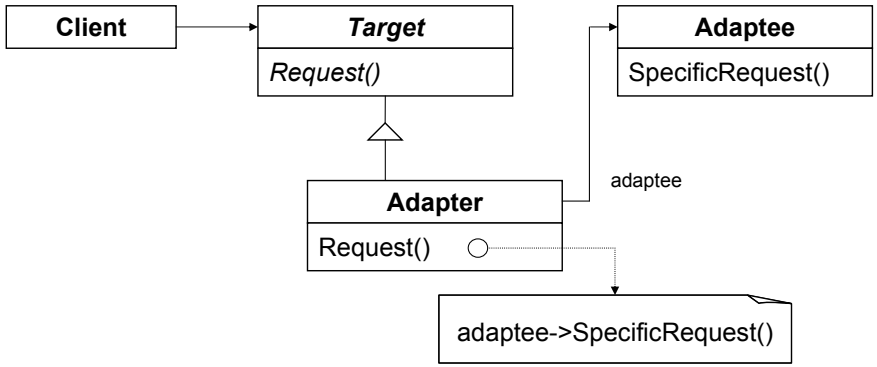
Vectors and Points

- Various implementations of Vectors by different modules
- How to map interface of reusable class into another interface client expects
- Strings, Lists and Dynamic Arrays

Adapter -- Structure



Object Adapter -- Structure



Adapter Consequences

- Object Adapter
  - + Lets single adapter work with entire Adaptee hierarchy
  - ± Adding functionality (to entire hierarchy) is straightforward
  - – Overriding functionality is difficult
- Pluggable Adapters
- Two-way adapters

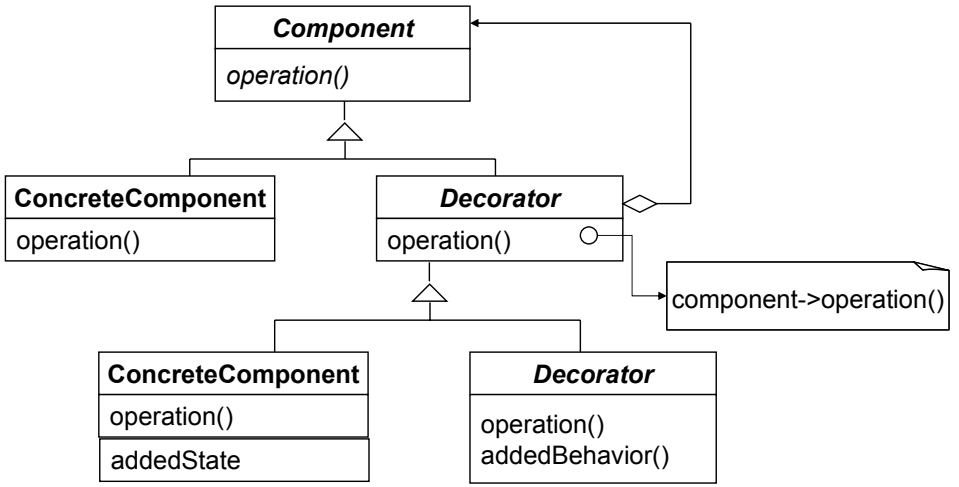
Border and Scrollbars

- Graphical user interface toolkit
- Adding decorations basic visual component like

  Border, Scrollbar, ...

- How to attach additional responsibilities to an object dynamically?

Decorator



Proxy

- Description
  - Defer the cost of object creation and initialization until actually used
- Applicability
  - Virtual Proxy (Lazy Construction)
  - Cache Proxy
  - Remote Proxy
  - Protection Proxy

Object Oriented Frameworks

Approaches to Practical Reuse

- Library routines
- Black box reuse: composition
- White box reuse: inheritance
- Toolkits: architecture independence
- Frameworks: rigid architecture
- Patterns: Problem, Context, & Solution

What is OO Framework?

- A set of related classes which you specialize and/or instantiate to implement an application or subsystem.
- A *reusable mini-architecture* that provides the generic structure and behavior for a family of abstractions, along with a context of metaphors which specifies their collaboration and use within a given domain.

Framework Characteristics

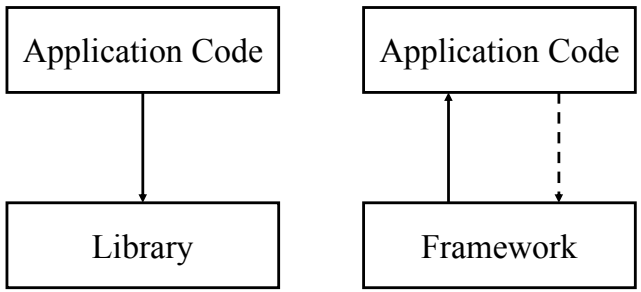
- “virtual machine" (or "virtual engine")
- open-ended abstractions
- plug-points (or hot spots)
- *a framework supplies the infrastructure and mechanisms that execute a policy for interaction between abstract components with open implementations*.

Framework and Class Library

- An OO Framework is a kind of class library, with a difference
- The difference is the flow control, which is bi-directional in case of frameworks

The Hollywood Principle

- “Don’t call us, we’ll call you.”



Framework and Architecture

- Framework dictates the architecture of your application.
- Framework captures the design decisions that are common to its application domain.
- Frameworks emphasize *design reuse* over code reuse.

Frameworks and Patterns

- *Design patterns are more abstract than frameworks*
- *Design patterns are smaller architectural elements than frameworks*
- *Design patterns are less specialized than frameworks*

Frameworks and Pattern Languages

- A pattern language describes **how to** make a design, while framework **is** a design

Framework Domains

- Application frameworks
  - Horizontal sections, e.g. GUI
- Domain frameworks
  - Vertical sections, control systems
- Support frameworks
  - system level services, such as file access, distributed computing support

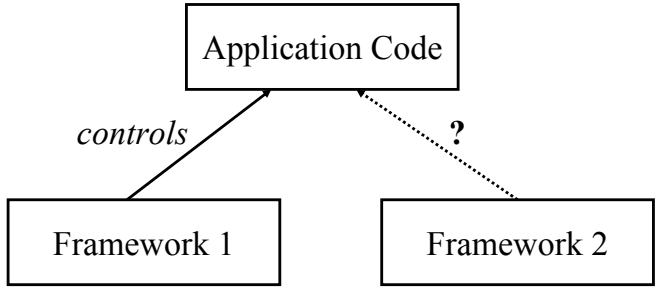
Advantages of Frameworks

- High reuse of design and code
- Bi-directional control flow can reduce coding efforts substantially
- Infrastructure and architectural guidance
- Mechanism for reliably extending functionality
- Reduce maintenance

Disadvantages of Frameworks

- Steep learning curve
- Loss of control due to the Hollywood principle
- No real good way to document frameworks
- Combining multiple frameworks is usually not easy

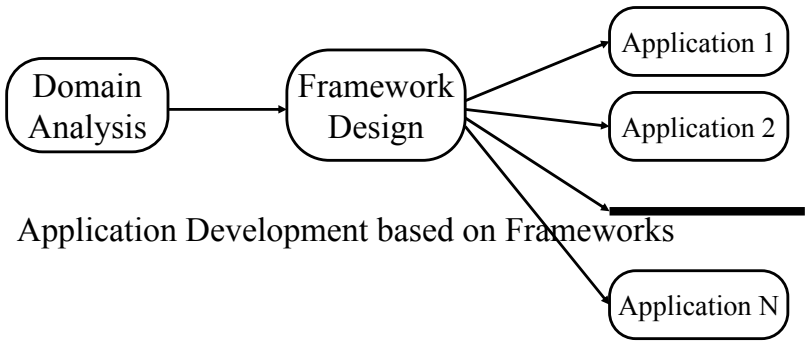
Framework Composition



Framework-centered Development

- The framework development phase
- The framework usage phase
- Framework evolution and maintenance phase

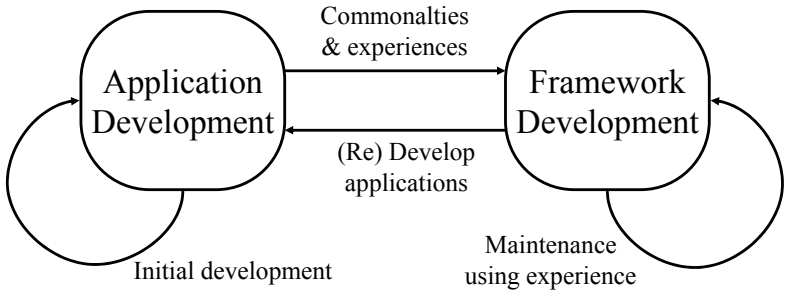
Traditional Object-Oriented Application Development



Application Development based on Frameworks

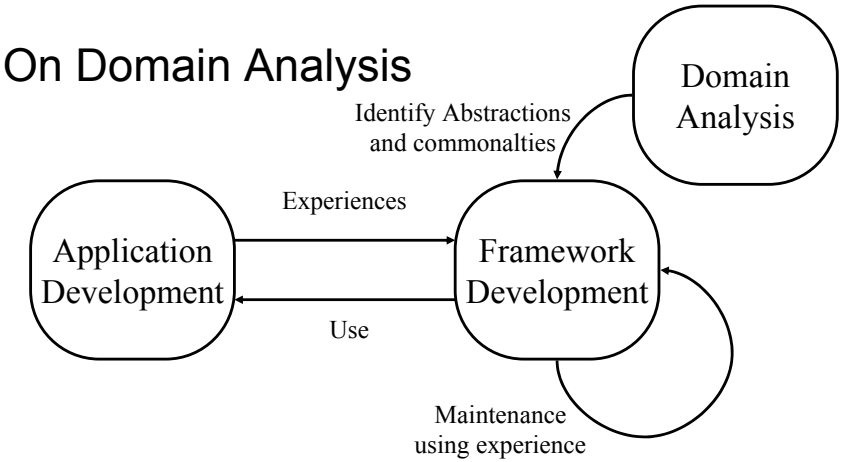
Developing Frameworks

- On Application Experiences



Developing Frameworks

- On Domain Analysis



Important Design Elements

- Abstract classes
  - Abstract superclass rule
- Dynamic binding
- Contracts
- Design Patterns

Evolving Frameworks

- White-box Framework
- Component Library
- Hot Spots
- Pluggable Objects
- Fine-grained Objects
- Black-box Framework

Evolving Frameworks

or Framework development made easy

- A Pattern language for developing frameworks
- Gives and guides a normal course of framework evolution

Pattern Name

- Context
- Problem
- Solution
- Forces
- Rationale

Three Examples

- You have decided to develop a framework for a problem domain
- How do you start designing?
- Develop three applications that you believe that framework should help you build and iteratively improve design
  - People develop abstractions from concrete examples

White-box Framework

- You have started to build second application
- Should framework rely on inheritance of polymorphic composition
- Use inheritance!
  - Inheritance retains flexibility
  - Not enough info to commit to composition

Component Library

- You are developing second and subsequent examples
- How to avoid coding similar objects for each application
- Start with simple library of objects and add objects as you need them
  - do not worry if problem specific objects are added to library at this stage

Hot Spots

- You are building component library
- You see similar code written over and over again (in same application)
- Separate code that changes and encapsulate it into objects
  - variation is achieved by composing rather than sub-classing and writing methods

Pluggable Objects

- You are building component library
- Most subclasses you write differ trivially
- Design adaptable subclasses that can be parameterized with messages to send, indexes to access, etc
  - New classes, no matter how trivial, increase complexity of the system

Fine-grained Objects

- You are re-factoring component library
- How far should you go in dividing objects into smaller objects?
- Break until it does not make sense to do so any further in the problem domain
  - It is important to avoid programming, tools can take care of proliferation of objects

Black-box Framework

- You are developing third application
- Inheritance or composition?
- Organize component library using inheritance and application using composition
  - Composition can be changed at runtime and is easier to use

Visual Builder

- You have a black-box framework
- How to simplify generation of connection scripts?
- Build a graphical tool that lets you specify the objects in your application and how they are connected
  - It is feasible and can be used by domain experts effectively to provide feedback

Framework Summary

- What is OO Framework?
- Framework characterization
- Development of Frameworks